Scikit-learn – consulta rápida

1 · Fluxo e preparação

Nível avançado · módulos N2_03 a N2_05
scikit-learn 1.6 ou superior

01 O FLUXO DE TRABALHO

N2_03 · introdução

1 Separar treino e teste, logo no início	**2 Preparar** escalar, codificar, imputar	**3 Treinar** `fit` só no treino
4 Comparar validação cruzada no treino	**5 Afinar** hiperparâmetros com CV	**6 Testar** uma única vez, no fim

02 SEPARAR OS DADOS

N2_03 · §3

`X = df[["area", "idade", "zona"]]` `y = df["preco"]`	X: variáveis de entrada (tabela). y: o que se quer prever.
`from sklearn.model_selection import (` `  train_test_split)`	Importar a função.

`X_tr, X_te, y_tr, y_te = (`
`  train_test_split(X, y,`
`  test_size=0.2, random_state=42))`
	80% treino · 20% teste, sempre a mesma divisão.

`train_test_split(..., stratify=y)`	Classificação: mantém as proporções das classes.

03 A API COMUM

N2_03 · §2

`modelo = LinearRegression()`	Criar o estimador.
`modelo.fit(X_tr, y_tr)`	Aprender com o treino.
`pred = modelo.predict(X_te)`	Prever dados novos.
`modelo.score(X_te, y_te)`	R^2 (regressão) ou exatidão (classificação).
`modelo.coef_` `modelo.intercept_`	O que foi aprendido: terminam em _.

04 PRÉ-PROCESSAMENTO

N2_03 · §5 · N2_04 · §4

`StandardScaler()`	Média 0 e desvio 1 por coluna.
`OneHotEncoder(handle_unknown="ignore")`	Categorias → colunas de 0 e 1.
`SimpleImputer(strategy="median")`	Preencher valores ausentes.
`PolynomialFeatures(degree=2)`	Termos quadráticos e interações.

`num = ["area", "idade"]`
`cat = ["zona"]`
`prep = ColumnTransformer([`
`  ("num", StandardScaler(), num),`
`  ("cat", OneHotEncoder(), cat)])`
	Tratar cada grupo de colunas de forma diferente.

05 PIPELINE

N2_03 · §5

`pipe = Pipeline([`
`  ("prep", prep),`
`  ("modelo", Ridge())])`
	Preparação e modelo num só objeto.

`pipe.fit(X_tr, y_tr)` `pipe.predict(X_te)`	Cada passo aprende só com o treino e aplica-se igual ao teste.
`pipe.named_steps["modelo"].coef_`	Aceder a um passo pelo nome.

`Pipeline([`
`  ("imputar", SimpleImputer()),`
`  ("escalar", StandardScaler())])`
	Encadear passos dentro de um grupo de colunas.

06 MÉTRICAS DE REGRESSÃO

N2_03 · §4

`mean_absolute_error(y_te, pred)`	MAE: erro médio, nas unidades de `y`.
`root_mean_squared_error(y_te, pred)`	RMSE: pesa mais os erros grandes.
`r2_score(y_te, pred)`	R^2: parte da variação explicada (1 = perfeito).
`DummyRegressor()`	Referência: prevê a média. O modelo tem de ser melhor.

07 VALIDAÇÃO E AFINAÇÃO

N2_03 · §7–10 · N2_04 · §6–7

`cv = cross_validate(pipe, X_tr, y_tr, cv=5, scoring="neg_mean_absolute_error")`	5 divisões do treino. Erros vêm negativos: maior é melhor.
`-cv["test_score"].mean()`	MAE médio da validação cruzada.
`StratifiedKFold(n_splits=5, shuffle=True, random_state=42)`	Divisões que mantêm as proporções das classes.
`busca = GridSearchCV(pipe, {"modelo__alpha": [0.1, 1, 10]}, cv=5)` `busca.fit(X_tr, y_tr)`	Testar combinações. Nome do passo + `__` + parâmetro.
`busca.best_params_` `busca.best_estimator_`	Melhor combinação · pipeline já reajustado.
`Ridge(alpha=1)` `Lasso(alpha=0.1)`	Regularização: `alpha` maior, coeficientes menores.

08 CLASSIFICADORES

N2_04 · §5 · N2_05 · §3

`LogisticRegression(max_iter=1000)`	Linear, dá probabilidades.
`DecisionTreeClassifier(max_depth=3)`	Regras fáceis de ler.
`KNeighborsClassifier(n_neighbors=5)`	Vizinhos mais próximos; escalar antes.
`RandomForestClassifier(n_estimators=300)`	Muitas árvores: robusto.
`HistGradientBoostingClassifier()`	Boosting rápido; aceita `NaN`.
`DummyClassifier(strategy="most_frequent")`	Referência: prevê a classe mais comum.

09 MÉTRICAS DE CLASSIFICAÇÃO

N2_04 · §3, §8

`confusion_matrix(y_te, pred)`	Acertos e erros de cada tipo.
`accuracy_score(y_te, pred)`	Exatidão: proporção de acertos.
`balanced_accuracy_score(y_te, pred)`	Média por classe: para classes desequilibradas.
`precision_score(y_te, pred)`	Dos sinalizados, quantos estavam certos.
`recall_score(y_te, pred)`	Dos casos reais, quantos foram apanhados.
`f1_score(y_te, pred)`	Equilíbrio entre precisão e recall.
`classification_report(y_te, pred)`	Todas as métricas por classe.
`roc_auc_score(y_te, proba)` `average_precision_score(y_te, proba)`	Qualidade da ordenação pelas probabilidades.

Scikit-learn – consulta rápida

3 · Decisões, aplicações e armadilhas

10 PROBABILIDADES E LIMIAR

N2_04 · §8–9 · N2_05 · §4

`proba = pipe.predict_proba(X_te)[:, 1]`	Probabilidade da classe positiva.
`pred = (proba >= 0.3).astype(int)`	Limiar escolhido pelo custo dos erros, não sempre 0,5.
`CalibratedClassifierCV( FrozenEstimator(pipe), method="sigmoid")`	Calibrar com dados à parte, sem voltar a treinar.

11 INTERPRETAR E VALIDAR MELHOR

N2_05 · §5–7

`permutation_importance(pipe, X_te, y_te, n_repeats=10)`	De que variáveis depende o modelo.
`GroupKFold(n_splits=5)` `cross_validate(..., groups=equipamento)`	O mesmo grupo nunca aparece no treino e na validação.
`TimeSeriesSplit(n_splits=5)`	Treinar com o passado, validar com o futuro.

12 SEM RÓTULOS

N2_05 · §8–10

`KMeans(n_clusters=3, random_state=42)`	Agrupar em k grupos (escalar antes).
`DBSCAN(eps=0.5, min_samples=5)`	Grupos por densidade; `-1` = ruído.
`PCA(n_components=2).fit_transform(X)`	Reduzir a 2 dimensões.
`pca.explained_variance_ratio_`	Variância guardada por componente.

13 GUARDAR O MODELO

N2_05 · §11

`joblib.dump(pipe, "modelo.joblib")`	Guarda o pipeline inteiro.
`pipe = joblib.load("modelo.joblib")`	Só abrir ficheiros de confiança.

! ARMADILHAS FREQUENTES

× errado · ✓ certo

Ajustar antes de separar. O teste «contamina» o treino.

✗ ~~`X = StandardScaler().fit_transform(X)`~~ ~~`train_test_split(X, y)`~~

✓ `train_test_split(X, y)` → `pipe.fit(X_tr, y_tr)`

Reajustar no teste. No teste só se aplica o que se aprendeu.

✗ ~~`scaler.fit_transform(X_te)`~~

✓ `scaler.transform(X_te)`

Avaliar no treino. Mede a memória, não a generalização.

✗ ~~`pipe.score(X_tr, y_tr)`~~

✓ `cross_validate(...)` · `pipe.score(X_te, y_te)`

Escolher pelo teste. O teste deixa de ser independente.

✗ ~~comparar 10 modelos no teste~~

✓ comparar com CV; testar uma vez

Classes desequilibradas. 95% de exatidão pode não valer nada.

✗ ~~`accuracy_score(y_te, pred)`~~

✓ `balanced_accuracy_score` · `recall` · `DummyClassifier`

Dividir ao acaso dados com tempo ou grupos. Resultados otimistas.

✗ ~~`KFold(shuffle=True)`~~

✓ `TimeSeriesSplit` · `GroupKFold`

Variáveis do futuro. Usar algo que só se sabe depois da previsão.

✗ ~~`X` inclui `"data_entrega"` para prever atrasos~~

✓ só variáveis conhecidas no momento da decisão